J-Manoj
192572143

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Measure network performance using key metrics with simulation tools. *(BL4,BL5)*

Assessment Tool Weightage: 30%

Annexure A

73½ 18

Pseudocode Writing Task (Algorithm Design)

1. In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics.
2. An organization has multiple network links connecting its branches. The administrator wants to monitor bandwidth utilization and identify overloaded links. Write a pseudocode algorithm that periodically collects the bandwidth usage of each network link and stores the values in an array or list. Use a loop to continuously monitor all links. Add conditions to detect when bandwidth utilization exceeds 80% and recommend switching traffic to a backup link. Explain how this algorithm supports efficient bandwidth management and prevents network congestion.
3. A company has six branch offices connected through multiple network links. Each link has different bandwidth, delay, and utilization values, which change throughout the day. The network administrator wants to continuously identify the best communication path between the headquarters and each

branch based on overall link quality rather than just the shortest distance.Write a pseudocode algorithm that periodically collects the bandwidth, delay, and utilization of every network link and stores the values in suitable arrays or lists. Use a loop to repeat the monitoring process at regular intervals. Calculate a link quality score for each path using the collected metrics and select the path with the highest score. Include conditions to detect when a link becomes overloaded or fails, automatically selecting an alternate path and generating an alert for the administrator. Explain how your algorithm improves network reliability, routing efficiency, and fault tolerance while adapting to changing network conditions.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

	A	B	C	D	E	
Q1	2	1	2	1	1	7
Q2	2	2	1.5	.5	1.5	7.5
Q3	1	3	1	.5	2	7.5

22/30

SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07

COURSE OUTCOME COVERED

CO5: Configure IP addressing schemes, subnetting, and basic routing techniques using simulation tool, for efficient network design and topology. *(BL3)*
Assessment Tool Weightage: 30%

QUESTIONS: 5 **Total Marks:50**

Annexure A

Code of Conduct:
I __j.manoj(192572143)__ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:
j.manoj

Question 1:

In a network simulation using Cisco Packet Tracer, a student records total data received as 8000 bits in 4 seconds and observes that 200 packets were sent while 180 packets were received. Write a pseudocode algorithm to calculate the network throughput in bits per second and the packet loss percentage based on the given values. The algorithm should include validation to ensure that time and total packets sent are not zero to avoid errors. It should display both the throughput and packet loss results clearly. Add logic to classify the network as Efficient if throughput is greater than 1500 bps and packet loss is less than 10%, otherwise classify it as Inefficient. Finally, ensure the algorithm outputs a complete summary of the network performance based on the calculated metrics. .

1. Problem Understanding

The task is to evaluate the performance of a simulated network link using two standard metrics — throughput and packet loss percentage — and to translate those numeric metrics into a qualitative efficiency verdict. Throughput measures how much data is successfully transferred per unit time, while packet loss percentage measures the reliability of delivery. A robust algorithm must not simply compute these values; it must first guard against invalid or degenerate inputs (division by zero when time or packets-sent is zero), then compute both metrics, then apply a two-condition classification rule, and finally present all of this as a single readable summary rather than isolated numbers.

2. Inputs and Outputs

Type	Name	Description	Given Value
Input	TotalDataReceived	Total data successfully received (bits)	8000
Input	Time	Duration of observation (seconds)	4
Input	PacketsSent	Total packets transmitted	200
Input	PacketsReceived	Total packets successfully received	180
Output	Throughput	Data transfer rate (bits per second)	Calculated
Output	PacketLossPercent	Percentage of packets lost in transit	Calculated
Output	NetworkStatus	"Efficient" or "Inefficient" classification	Calculated
Output	PerformanceSummary	Consolidated report of all metrics	Calculated

3. Assumptions and Constraints

- All input values (TotalDataReceived, Time, PacketsSent, PacketsReceived) are non-negative numbers supplied by the simulation environment.

- Time and PacketsSent are the two values that can legitimately cause a division-by-zero error, so both must be validated before any arithmetic is performed.
- PacketsReceived is assumed to never exceed PacketsSent under normal simulation behaviour.
- Throughput is measured strictly in bits per second (bps); no unit conversion to Kbps/Mbps is required unless requested.
- The efficiency thresholds (throughput > 1500 bps AND packet loss < 10%) are fixed constants defined by the assessment, and the classification uses a strict AND — both conditions must hold for an "Efficient" verdict.

4. Detailed Algorithm Design

The algorithm follows five stages: (1) Input acquisition, (2) Validation of denominators, (3) Metric computation, (4) Rule-based classification, and (5) Summary generation and display. Validation is placed immediately after input capture so that no downstream computation ever executes on an invalid state. Throughput is derived by dividing total received data by elapsed time. Packet loss is derived from the proportion of packets that were sent but never received, expressed as a percentage. The classification stage applies a compound Boolean condition, and the final stage assembles every computed value into one structured report.

Step	Stage	Purpose
1	Input Acquisition	Read TotalDataReceived, Time, PacketsSent, PacketsReceived
2	Validation	Reject the run if Time = 0 or PacketsSent = 0
3	Throughput Computation	Throughput = TotalDataReceived / Time
4	Packet Loss Computation	PacketLoss% = ((PacketsSent − PacketsReceived) / PacketsSent) × 100
5	Classification	IF Throughput > 1500 AND PacketLoss% < 10 → Efficient, ELSE Inefficient
6	Summary & Display	Print throughput, packet loss, and final status as one report

5. Well-Structured Pseudocode

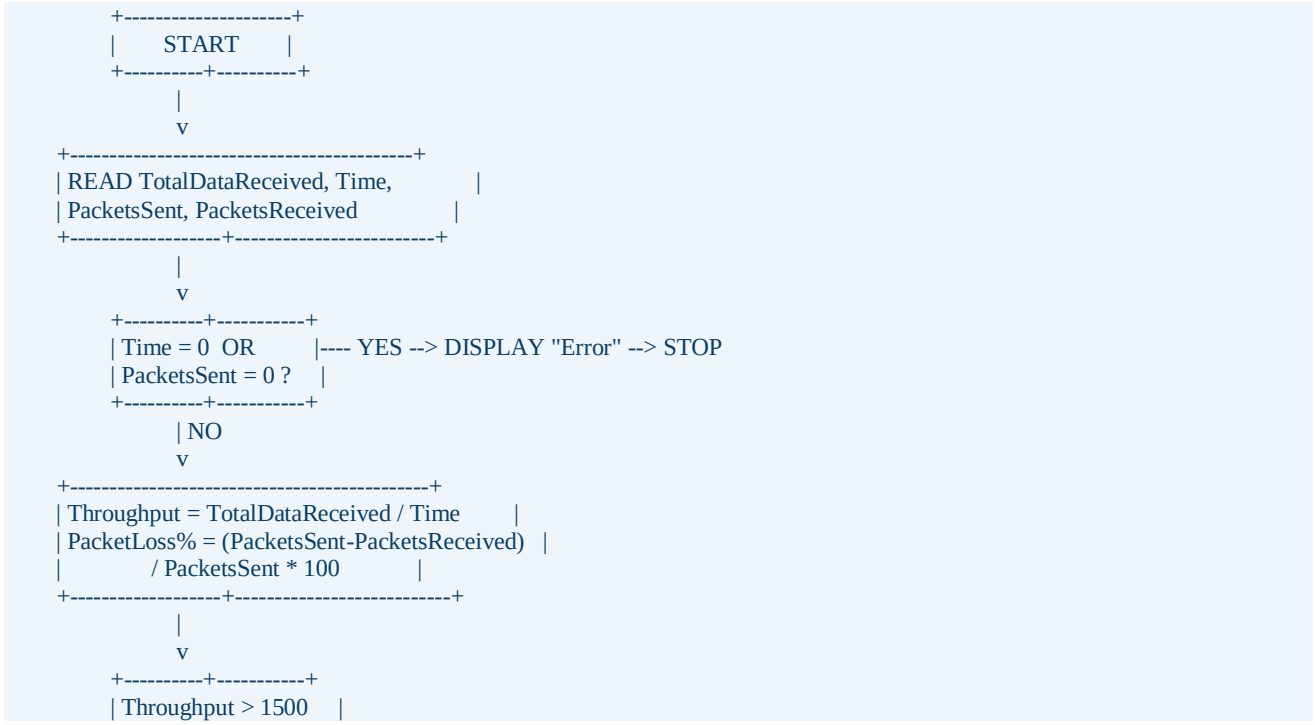
```
ALGORITHM CalculateThroughputAndPacketLoss
BEGIN
   // Step 1: Input
   READ TotalDataReceived       // in bits
   READ Time                    // in seconds
   READ PacketsSent
   READ PacketsReceived

   // Step 2: Validation
   IF Time = 0 OR PacketsSent = 0 THEN
      DISPLAY "Error: Time and PacketsSent must not be zero."
      STOP
   END IF

   // Step 3: Throughput calculation
   Throughput = TotalDataReceived / Time        // bits per second

   // Step 4: Packet loss calculation
   PacketsLost = PacketsSent - PacketsReceived
```

```
    PacketLossPercent = (PacketsLost / PacketsSent) * 100

    // Step 5: Classification
    IF Throughput > 1500 AND PacketLossPercent < 10 THEN
        NetworkStatus = "Efficient"
    ELSE
        NetworkStatus = "Inefficient"
    END IF

    // Step 6: Display summary
    DISPLAY "----- Network Performance Summary -----"
    DISPLAY "Throughput       : " + Throughput + " bps"
    DISPLAY "Packet Loss      : " + PacketLossPercent + " %"
    DISPLAY "Network Status   : " + NetworkStatus
    DISPLAY "---------------------------------------"
END
```

6. Explanation of Each Step

- Step 1 (Input): The four raw measurements collected by the simulator are read into named variables so the rest of the algorithm can reference them symbolically.
- Step 2 (Validation): A guard clause checks both denominators used later (Time and PacketsSent). If either is zero, the algorithm halts immediately with an error message instead of allowing an undefined division.
- Step 3 (Throughput): Dividing the total bits received by the elapsed time gives the average data rate in bits per second — the standard definition of throughput.
- Step 4 (Packet Loss): The number of packets lost is the difference between sent and received; dividing by the total sent and multiplying by 100 converts this into a percentage.
- Step 5 (Classification): A compound condition (logical AND) ensures the network is only labelled "Efficient" when it is simultaneously fast and reliable — a single good metric is not sufficient.
- Step 6 (Summary): All computed values and the final verdict are displayed together in one formatted block so the result is easy to read and interpret.

7. Flowchart (Text Format)



8. Sample Input and Output

TotalDataReceived (bits)	Time (s)	PacketsSent	PacketsReceived
8000	4	200	180

```
----- Network Performance Summary -----
Throughput       : 2000 bps
Packet Loss      : 10 %
Network Status   : Inefficient
----------------------------------------
```

9. Working Example with Calculations

Metric	Formula	Substitution	Result
Throughput	TotalDataReceived / Time	8000 / 4	2000 bps
Packets Lost	PacketsSent − PacketsReceived	200 − 180	20 packets
Packet Loss %	(PacketsLost / PacketsSent) × 100	(20 / 200) × 100	10 %

Classification check: Throughput = 2000 bps, which satisfies Throughput > 1500 (TRUE). However, Packet Loss = 10%, and the condition requires Packet Loss < 10%, which is FALSE because 10 is not strictly less than 10. Since the classification rule uses a logical AND, both conditions must be TRUE simultaneously; here one condition fails, so the overall result is NetworkStatus = "Inefficient", even though the throughput alone looks acceptable. This boundary case demonstrates why the algorithm must evaluate both metrics together rather than relying on throughput alone.

10. Advantages of the Algorithm

- Prevents runtime crashes by validating denominators before any division is performed.
- Uses simple, well-known formulas that map directly to standard networking definitions of throughput and packet loss.
- Combines two independent metrics into a single, meaningful classification rather than leaving the administrator to interpret raw numbers.
- Produces a clear, structured summary that is easy to log, compare across simulation runs, or export to a report.
- Is lightweight (O(1) time and space complexity) and can be executed repeatedly for continuous or batch evaluation.

11. Network Performance Analysis

With 8000 bits delivered in 4 seconds, the link sustains 2000 bps, which is comfortably above the 1500 bps efficiency benchmark and indicates adequate raw capacity for the observed load. However, 20 of 200 packets (10%) failed to arrive, exactly at the reliability threshold. In real network terms, a 10% loss rate is high enough to noticeably degrade interactive applications (voice, video, real-time control traffic) even if the raw bit rate is acceptable, because retransmissions and jitter compound the effective delay. The algorithm's strict-AND classification correctly flags this link as "Inefficient" overall, steering attention toward reliability (congestion, faulty links, buffer overflows) rather than raw capacity.

12. Conclusion

The designed algorithm reliably computes throughput and packet loss from raw simulation counters, guards against invalid inputs, and applies a dual-condition rule to classify network efficiency. Applied to the given data, it correctly identifies that a network can appear fast (2000 bps) yet still be classified "Inefficient" once packet loss reaches the 10% boundary, illustrating the importance of evaluating throughput and reliability together rather than in isolation.

Question 2:

An organization has multiple network links connecting its branches. Design a pseudocode algorithm that periodically collects bandwidth utilization for each link, stores the values in arrays/lists, continuously monitors all links in a loop, detects when utilization exceeds 80%, and recommends switching traffic to a backup link. Explain how the algorithm supports efficient bandwidth management and prevents congestion.

1. Problem Understanding

The organization needs an always-on monitoring process rather than a one-time calculation. The algorithm must repeatedly sample bandwidth utilization across an arbitrary number of links, maintain those readings in indexed data structures, compare each reading against a fixed overload threshold (80%), and issue an actionable recommendation (switch to backup) whenever a link crosses that threshold. Because the process is continuous, the design must use a monitoring loop with a defined sampling interval rather than a single pass.

2. Inputs and Outputs

Type	Name	Description
Input	NumberOfLinks	Total number of monitored network links
Input	LinkNames[]	Identifier for each link (e.g., L1, L2, ...)
Input	BandwidthUtilization[]	Current utilization percentage sampled per link, per interval
Input	OverloadThreshold	Fixed limit for overload detection (80%)
Input	SamplingInterval	Time (seconds) between successive monitoring cycles
Output	UtilizationLog[][]	Stored history of utilization readings for each link
Output	OverloadAlerts[]	List of links currently exceeding the threshold
Output	SwitchRecommendation[]	Backup-link switch action recommended per overloaded link

3. Assumptions and Constraints

- Every monitored link has one designated backup link available for failover traffic.
- Bandwidth utilization is expressed as a percentage (0–100) of the link's maximum rated capacity.
- The overload threshold is a fixed constant (80%) applied uniformly to all links.
- The monitoring loop runs indefinitely (or until explicitly stopped by the administrator) and re-samples at a fixed SamplingInterval.
- Utilization values are stored (not discarded) after each cycle so that historical trends can be reviewed.

4. Detailed Algorithm Design

The algorithm initializes an array to hold the list of links and a parallel array (or 2-D log) to accumulate utilization readings over time. A continuous outer loop represents the periodic monitoring cycle; within each cycle, an inner loop iterates over every link, samples its current utilization, appends the reading to the log, and immediately checks it against the 80% threshold. Any link exceeding the threshold is added to an overload list and a backup-switch recommendation is generated and displayed for that link. After the inner loop completes, the algorithm waits for SamplingInterval before beginning the next cycle.

Step	Stage	Purpose
1	Initialization	Define link list, empty utilization log, threshold = 80%
2	Outer Loop (WHILE TRUE)	Represents continuous, periodic monitoring
3	Inner Loop (FOR each link)	Iterate through every link in the current cycle
4	Sampling	Read current bandwidth utilization for the link
5	Logging	Append the reading to the link's utilization history
6	Overload Check	IF utilization > 80% THEN flag link and recommend backup switch
7	Wait	Pause for SamplingInterval before the next cycle

5. Well-Structured Pseudocode

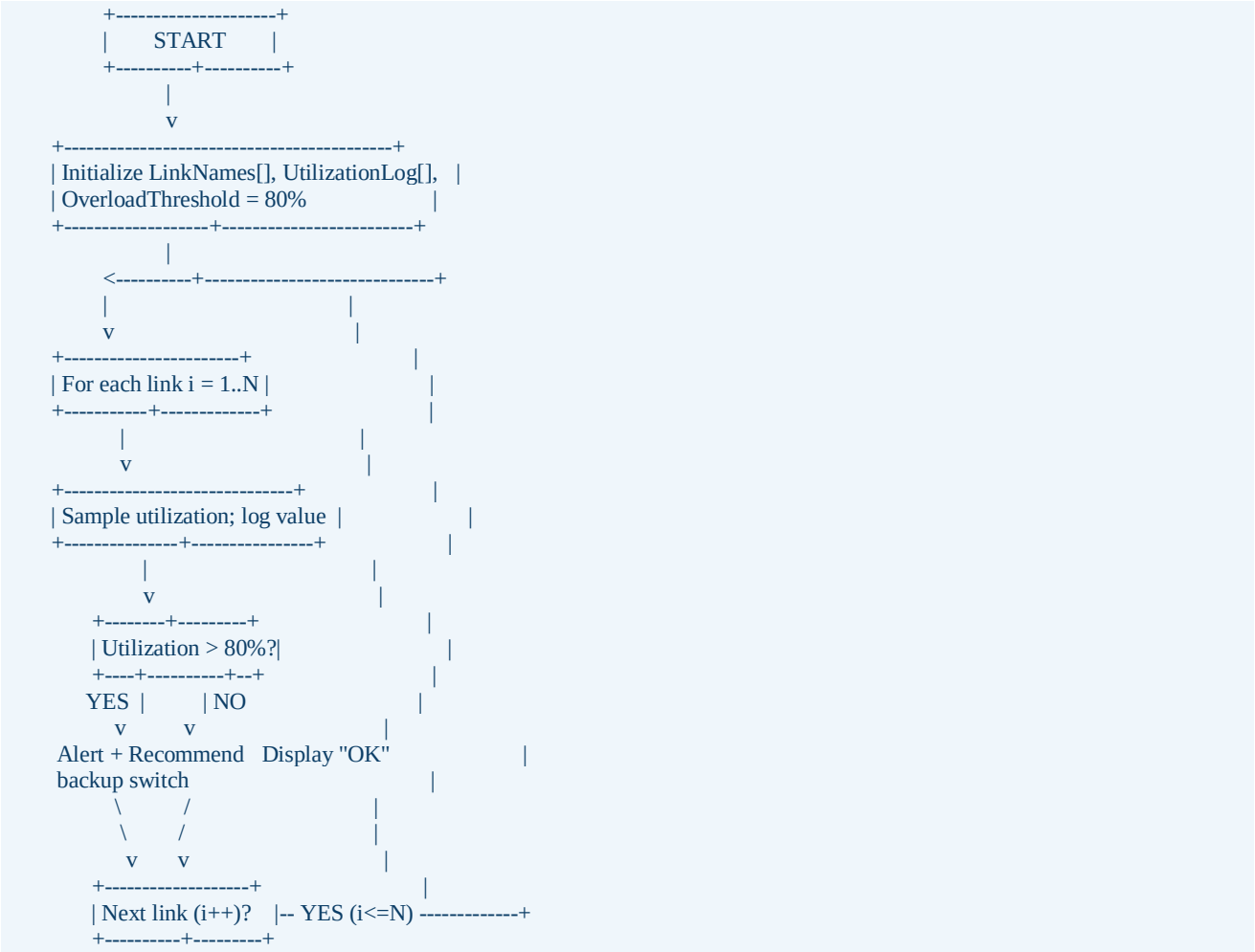
```
ALGORITHM MonitorBandwidthUtilization
BEGIN
   // Step 1: Initialization
   DEFINE LinkNames[N]                    // e.g., L1..LN
   DEFINE UtilizationLog[N][]  = EMPTY       // history per link
   OverloadThreshold = 80                 // percent
   SamplingInterval  = T                  // seconds

   // Step 2: Continuous monitoring loop
   WHILE (MonitoringActive = TRUE) DO

      // Step 3: Check every link in this cycle
      FOR i = 1 TO N DO

         // Step 4: Sample current utilization
         CurrentUtilization = READ_BANDWIDTH(LinkNames[i])

         // Step 5: Store reading
         APPEND CurrentUtilization TO UtilizationLog[i]

         // Step 6: Overload detection
         IF CurrentUtilization > OverloadThreshold THEN
            DISPLAY "ALERT: " + LinkNames[i] + " overloaded at " +
                  CurrentUtilization + "%"
            DISPLAY "Recommendation: Switch traffic on " + LinkNames[i] +
                  " to its backup link"
            SwitchTrafficToBackup(LinkNames[i])
         ELSE
            DISPLAY LinkNames[i] + " OK (" + CurrentUtilization + "%)"
         END IF

      END FOR
```

```
    // Step 7: Wait before next monitoring cycle
    WAIT (SamplingInterval)

  END WHILE
END
```

6. Explanation of Each Step

- Initialization: Establishes the list of links, an empty per-link history array, and the fixed 80% overload threshold, giving the algorithm a known starting state.
- Outer WHILE loop: Models the fact that monitoring is a continuous, never-ending background process, not a single check.
- Inner FOR loop: Ensures every link — regardless of how many exist — is checked exactly once per cycle, making the algorithm scale to any number of branches.
- Sampling and Logging: Each reading is both used immediately (for the threshold check) and preserved in UtilizationLog, enabling later trend analysis or capacity planning.
- Overload Check and Recommendation: The IF/ELSE construct separates normal links from overloaded ones and triggers a concrete, actionable recommendation (switch to backup) only when needed, avoiding unnecessary alerts.
- Wait/Interval: Pacing the loop with SamplingInterval prevents the algorithm from consuming excessive processing resources while still monitoring frequently enough to catch overload conditions promptly.

7. Flowchart (Text Format)



```
        | NO (all links done)
        v
+-----------------+
| WAIT SamplingIvl |
+---------+---------+
        |
        v
(loop back to "For each link")
```

8. Sample Input and Output

Assume 5 links are monitored in one cycle with the following sampled utilization readings:

Link	Utilization (%)	Status
L1	65	OK
L2	82	ALERT – Overloaded
L3	91	ALERT – Overloaded
L4	45	OK
L5	78	OK

```
L1 OK (65%)
ALERT: L2 overloaded at 82%
Recommendation: Switch traffic on L2 to its backup link
ALERT: L3 overloaded at 91%
Recommendation: Switch traffic on L3 to its backup link
L4 OK (45%)
L5 OK (78%)
```

9. Working Example with Calculations

The overload check is a direct comparison rather than a derived formula: for each link, CurrentUtilization is compared against OverloadThreshold = 80. For L2, 82 > 80 is TRUE, so L2 is flagged and a backup switch is recommended. For L3, 91 > 80 is TRUE, triggering the same response. For L1 (65), L4 (45), and L5 (78), the condition is FALSE, so each is reported as OK. Tracking this across three consecutive monitoring cycles illustrates how the log builds a utilization trend

Link	Cycle 1 (%)	Cycle 2 (%)	Cycle 3 (%)	Trend
L1	65	68	70	Rising, still safe
L2	82	85	79	Overloaded twice, recovered
L3	91	88	84	Persistently overloaded
L4	45	50	48	Stable, safe
L5	78	83	81	Crossed threshold in cycles 2–3

10. Advantages of the Algorithm

- Continuous WHILE-loop monitoring detects overload conditions in near real time rather than only during manual checks.
- Per-link FOR-loop iteration scales automatically to any number of branches or links without modifying the algorithm's structure.
- Immediate, targeted recommendations (switch to backup) turn raw percentages into actionable network-management decisions.
- Historical logging (UtilizationLog) supports capacity planning and trend analysis, not just instantaneous alerts.
- A fixed sampling interval balances monitoring responsiveness against system/processing overhead.

11. Network Performance Analysis

In the sample cycle, 2 of 5 links (40%) exceed the 80% utilization threshold, indicating that nearly half the monitored infrastructure is at risk of congestion-induced packet loss, increased latency, and queuing delay. L3's persistence above threshold across all three cycles marks it as a chronic bottleneck requiring either a permanent backup activation or a capacity upgrade, whereas L5's single-cycle crossing suggests a transient traffic spike that self-resolves. By automatically recommending backup switching the moment the 80% line is crossed, the algorithm shifts load away from saturated links before they degrade into packet loss, directly supporting the organization's goal of proactive rather than reactive congestion management.

12. Conclusion

The designed algorithm continuously samples, logs, and evaluates bandwidth utilization across all organizational links using nested loop control structures. By comparing each reading against an 80% threshold and immediately recommending a backup-link switch for any overloaded link, it converts raw monitoring data into timely, automated congestion-prevention actions, supporting efficient and resilient bandwidth management across the branch network.

QUESTION 3:

A company has six branch offices connected through multiple network links, each with changing bandwidth, delay, and utilization values. Design a pseudocode algorithm that periodically collects these three metrics for every link, computes a link-quality score, selects the highest-scoring path between headquarters and each branch, detects overloaded or failed links, automatically switches to an alternate path, and generates an administrator alert. Explain how the algorithm improves reliability, routing efficiency, and fault tolerance.

1. Problem Understanding

Unlike Questions 1 and 2, this problem requires combining three metrics — bandwidth, delay, and utilization — into a single composite score so that paths can be ranked by overall quality rather than by any one metric or by shortest distance alone. It also introduces fault tolerance: the algorithm must recognize when the currently selected path degrades (overloaded or failed) and autonomously reroute traffic to a pre-identified alternate path, notifying the administrator whenever this happens. Because link conditions change throughout the day, the process must run continuously, just as in Question 2, but with a richer decision rule.

2. Inputs and Outputs

Type	Name	Description
Input	Branches[]	Six branch office identifiers (B1..B6)
Input	CandidatePaths[branch][]	Primary and backup link(s) available to each branch
Input	Bandwidth, Delay, Utilization	Per-link metrics sampled every monitoring cycle
Input	W1, W2, W3	Weighting factors for bandwidth, delay, and utilization
Input	OverloadThreshold	Utilization limit beyond which a link is considered overloaded (80%)
Output	LinkQualityScore[]	Composite score computed for each candidate link
Output	SelectedPath[branch]	Highest-scoring, non-overloaded path chosen per branch
Output	AdminAlerts[]	Alerts raised when a link is overloaded/failed and rerouted

3. Assumptions and Constraints

- Each of the six branches has at least two candidate links (one Primary, one Backup) between it and headquarters.
- Higher bandwidth improves quality (positive contribution); higher delay and higher utilization reduce quality (negative contributions).

- The composite Link Quality Score uses fixed weights: W1 = 0.5 for bandwidth, W2 = 0.3 for delay, W3 = 0.2 for utilization, reflecting that raw capacity matters most, delay next, and current load least, for ranking purposes.
- A link with utilization above 80% is treated as overloaded/unusable for selection purposes, regardless of its raw score, and a link with no response is treated as failed.
- The monitoring loop re-evaluates all branches at a fixed interval, since bandwidth, delay, and utilization vary throughout the day.

4. Detailed Algorithm Design

For every monitoring cycle, the algorithm loops over each of the six branches. For each branch, it loops over the branch's candidate links (Primary, Backup, …), samples Bandwidth, Delay, and Utilization for each, and computes a Link Quality Score using the weighted formula Score = (W1 × Bandwidth) − (W2 × Delay) − (W3 × Utilization). Any link whose Utilization exceeds the OverloadThreshold — or that fails to respond — is excluded from selection and triggers an administrator alert. Among the remaining (healthy) candidates, the algorithm selects the link with the maximum score as SelectedPath for that branch. If the branch's previously active path is no longer the selected one, the algorithm performs an automatic switch and logs the change.

Step	Stage	Purpose
1	Initialization	Define branches, candidate links per branch, weights, threshold
2	Outer Loop (WHILE TRUE)	Continuous periodic monitoring
3	Branch Loop (FOR each branch)	Evaluate every one of the six branches each cycle
4	Link Loop (FOR each candidate link)	Sample metrics for Primary, Backup, etc.
5	Failure/Overload Check	Exclude link if Utilization > 80% or link is unresponsive; raise alert
6	Score Computation	Score = W1×Bandwidth − W2×Delay − W3×Utilization
7	Best-Path Selection	Choose the healthy link with the highest score
8	Failover Switch	If selection changed from current path, switch and log the event
9	Wait	Pause for SamplingInterval before the next cycle

5. Well-Structured Pseudocode

```
ALGORITHM SelectBestCommunicationPath
BEGIN
   // Step 1: Initialization
   DEFINE Branches[6]                    // B1..B6
   DEFINE CandidateLinks[branch][]       // Primary, Backup, ...
   W1 = 0.5   // weight: bandwidth
   W2 = 0.3   // weight: delay
   W3 = 0.2   // weight: utilization
   OverloadThreshold = 80
   SamplingInterval  = T
```

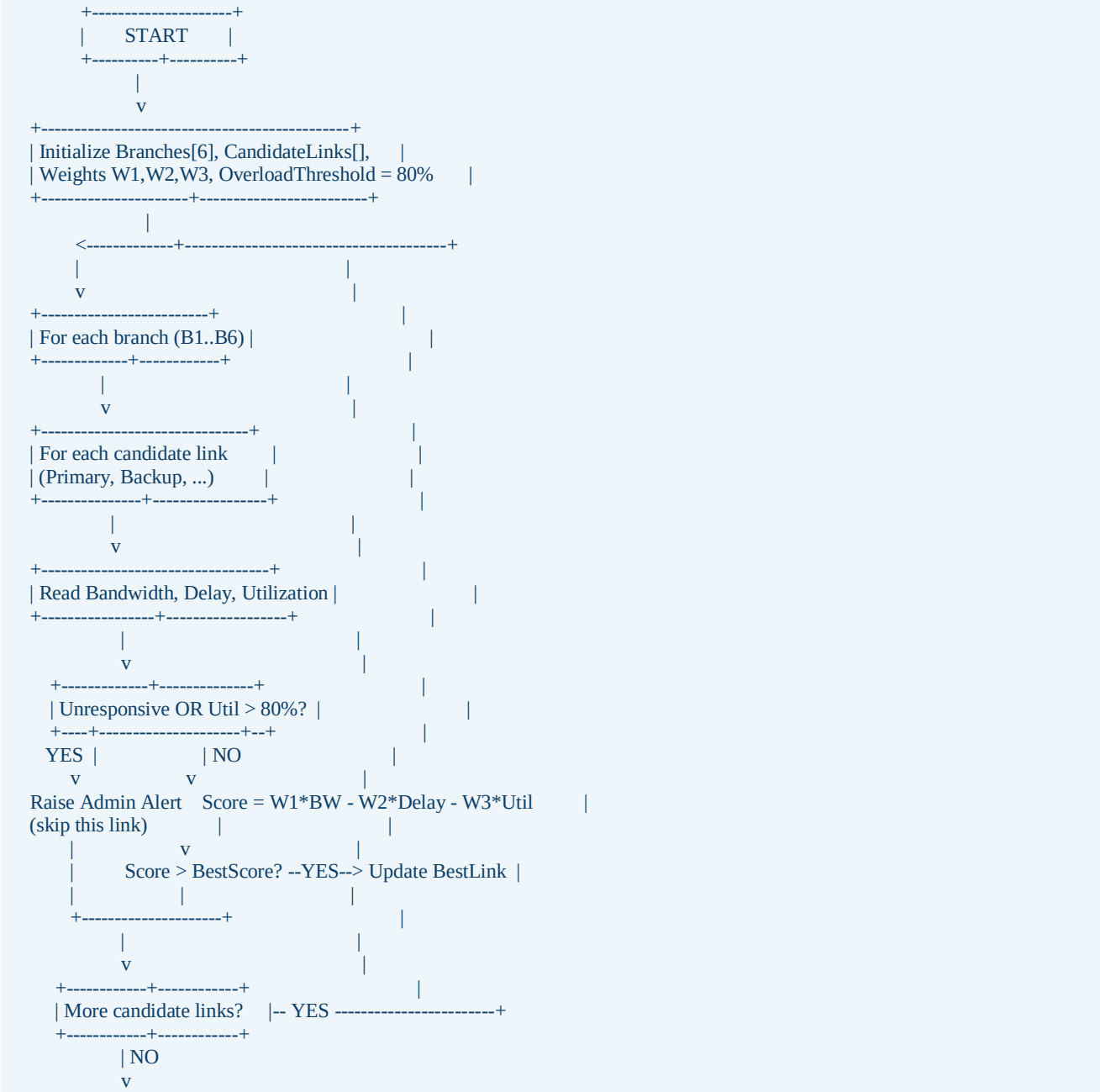
```
// Step 2: Continuous monitoring loop
WHILE (MonitoringActive = TRUE) DO

   // Step 3: Evaluate every branch
   FOR EACH branch IN Branches DO

      BestScore = -INFINITY
      BestLink  = NULL

      // Step 4: Evaluate every candidate link for this branch
      FOR EACH link IN CandidateLinks[branch] DO

         Bandwidth   = READ_BANDWIDTH(link)
         Delay       = READ_DELAY(link)
         Utilization = READ_UTILIZATION(link)

         // Step 5: Failure / overload check
         IF (link IS UNRESPONSIVE) OR (Utilization > OverloadThreshold) THEN
            DISPLAY "ALERT: " + link + " for " + branch +
                  " is overloaded/failed (Util=" + Utilization + "%)"
            CONTINUE                  // skip scoring this link
         END IF

         // Step 6: Composite link quality score
         Score = (W1 * Bandwidth) - (W2 * Delay) - (W3 * Utilization)

         // Step 7: Track the best healthy candidate
         IF Score > BestScore THEN
            BestScore = Score
            BestLink  = link
         END IF

      END FOR

      // Step 8: Apply selection / failover
      IF BestLink = NULL THEN
         DISPLAY "CRITICAL ALERT: No healthy path available for " + branch
      ELSE IF BestLink <> CurrentPath[branch] THEN
         DISPLAY "Switching " + branch + " from " + CurrentPath[branch] +
               " to " + BestLink + " (Score=" + BestScore + ")"
         CurrentPath[branch] = BestLink
      ELSE
         DISPLAY branch + " remains on " + BestLink +
               " (Score=" + BestScore + ")"
      END IF

   END FOR

   // Step 9: Wait before next monitoring cycle
   WAIT (SamplingInterval)

END WHILE
END
```

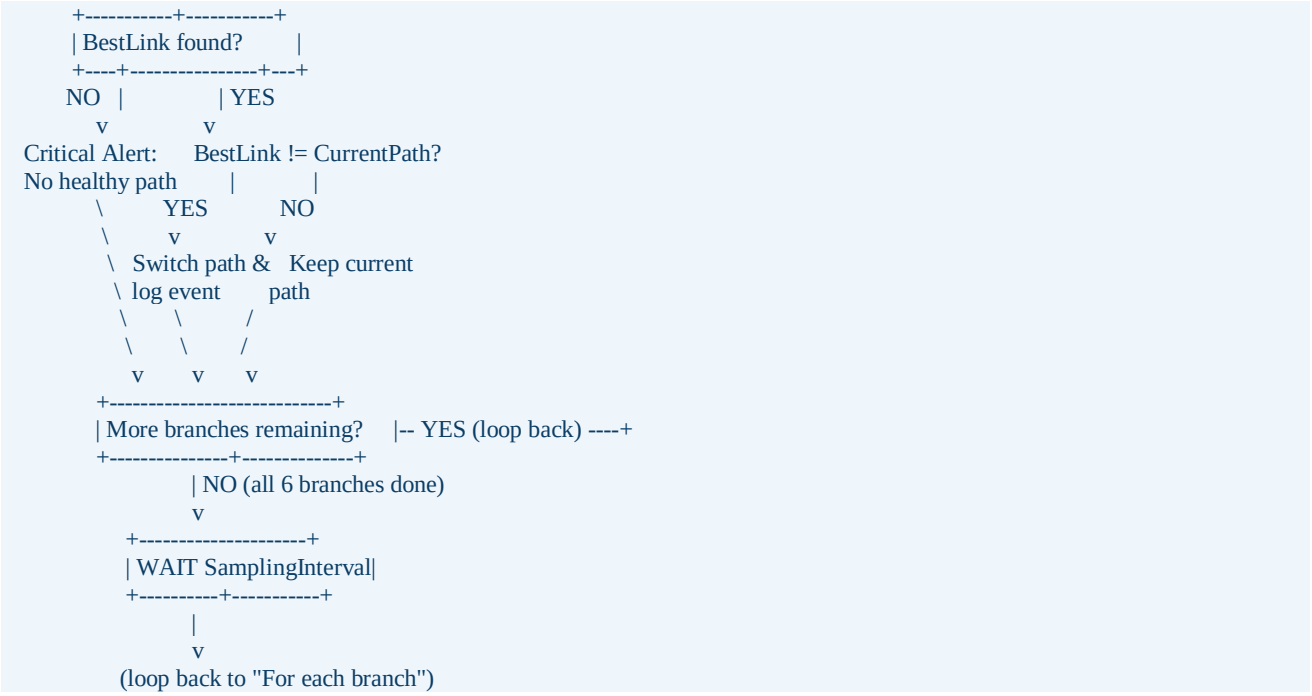
6. Explanation of Each Step

- Initialization: Fixes the branch list, candidate-link structure, weighting scheme (0.5 / 0.3 / 0.2), and overload threshold before monitoring begins.
- Outer WHILE loop: Keeps path selection current as bandwidth, delay, and utilization change throughout the day, rather than computing it once.

- Branch FOR loop: Guarantees all six branches are re-evaluated every cycle, independent of one another.
- Link FOR loop: Considers every candidate link (Primary, Backup, …) for a branch so the algorithm always has a fallback to compare against.
- Failure/Overload Check: Immediately disqualifies unhealthy links (CONTINUE skips scoring), and raises an administrator alert before that link can ever be selected.
- Score Computation: The weighted formula rewards high bandwidth and penalizes high delay and high utilization, producing a single number that reflects overall path quality rather than one dimension.
- Best-Path Tracking: A running BestScore/BestLink comparison across the candidate loop finds the maximum-quality healthy link without needing to sort the full list.
- Selection/Failover: Comparing BestLink to the branch's CurrentPath means the algorithm only logs a switch when the path actually changes, and explicitly raises a critical alert if no healthy path remains at all.

7. Flowchart (Text Format)





8. Sample Input and Output

Score formula: Score = (0.5 × Bandwidth) − (0.3 × Delay) − (0.2 × Utilization). Bandwidth is in Mbps, Delay in ms, Utilization in %. Sample readings for one monitoring cycle across all six branches (Primary and Backup candidate for each):

Branch	Link	BW (Mbps)	Delay (ms)	Util (%)	Score	Status
B1	Primary	100	20	40	36.0	Healthy
B1	Backup	70	25	35	20.5	Healthy
B2	Primary	80	15	85	18.5	Overloaded
B2	Backup	70	18	50	19.6	Healthy
B3	Primary	120	25	70	38.5	Healthy
B3	Backup	90	20	55	28.0	Healthy
B4	Primary	60	10	30	21.0	Healthy
B4	Backup	65	12	35	21.9	Healthy
B5	Primary	150	35	90	46.5	Overloaded
B5	Backup	100	30	55	30.0	Healthy
B6	Primary	90	18	50	29.6	Healthy
B6	Backup	85	20	45	27.5	Healthy

9. Working Example with Calculations

Sample calculation for Branch 1, Primary link: Bandwidth = 100 Mbps, Delay = 20 ms, Utilization = 40%.

```
Score(B1-Primary) = (0.5 x 100) - (0.3 x 20) - (0.2 x 40)
                  = 50 - 6 - 8
```

= 36.0

Sample calculation for Branch 5, Primary link, which is overloaded: Bandwidth = 150 Mbps, Delay = 35 ms, Utilization = 90%.

```
Score(B5-Primary) = (0.5 x 150) - (0.3 x 35) - (0.2 x 90)
                  = 75 - 10.5 - 18
                  = 46.5   <- highest raw score, but Utilization
                           90% > 80% threshold => DISQUALIFIED
ALERT: B5-Primary overloaded/failed -> excluded from selection
Best healthy candidate for B5 = Backup (Score = 30.0)
SWITCH: B5 rerouted from Primary to Backup
```

Resulting best-path selection per branch after applying the overload filter and maximum-score rule:

Branch	Selected Path	Score	Reason
B1	Primary	36.0	Higher score than Backup (20.5); healthy
B2	Backup	19.6	Primary overloaded (85% > 80%); Backup healthy and scores higher
B3	Primary	38.5	Higher score than Backup (28.0); healthy
B4	Backup	21.9	Slightly higher score than Primary (21.0); both healthy
B5	Backup	30.0	Primary overloaded (90% > 80%) despite highest raw score (46.5)
B6	Primary	29.6	Higher score than Backup (27.5); healthy

10. Advantages of the Algorithm:

- Combines three independent metrics (bandwidth, delay, utilization) into one composite score, enabling genuinely quality-based path selection instead of shortest-distance or single-metric routing.
- Automatically disqualifies overloaded or failed links before scoring, preventing the algorithm from ever recommending an unhealthy path — as shown by B5, where the highest-scoring link was correctly rejected.
- Per-branch, per-link nested loops scale cleanly to any number of branches or candidate links without redesign.
- Failover only triggers a logged switch when the selected link actually changes, avoiding unnecessary churn while still reacting immediately when conditions change.
- Administrator alerts (per-link overload and critical no-healthy-path conditions) give operators visibility and an audit trail of every automatic rerouting decision.

11. Network Performance Analysis

Across the six branches, two Primary links (B2 and B5) are overloaded in this cycle, forcing automatic failover to their Backup links; the remaining four branches stay on their higher-scoring Primary link. Branch 5 is the most instructive case: its Primary link has the single highest raw quality score (46.5) of any link in the network, yet its 90% utilization places it well past the 80% overload threshold, so the algorithm correctly excludes it and falls back to a lower-but-healthy Backup (score 30.0). This

demonstrates that raw capacity and delay alone are insufficient for routing decisions — current load must gate eligibility before scoring is even considered. Branch 4 shows the opposite pattern: both candidate links are healthy, and the algorithm's fine-grained scoring picks the marginally better Backup (21.9 vs 21.0), a distinction a simpler 'shortest path' rule would likely miss entirely.

12. Conclusion

The designed algorithm continuously monitors bandwidth, delay, and utilization for every candidate link across all six branches, computes a weighted link-quality score, and selects the best available healthy path while automatically failing over and alerting the administrator whenever a link becomes overloaded or unresponsive. By gating selection on link health before ranking by score — as demonstrated when Branch 5's highest-scoring Primary link was correctly overridden by a healthy Backup — the algorithm materially improves routing efficiency, network reliability, and fault tolerance, while adapting automatically to conditions that change throughout the day.